

docker

Test de repaso — parte 1: fundamentos de Docker

100 preguntas · 4 opciones, 1 correcta

Cómo usar este banco de preguntas

- Cada diapositiva contiene una pregunta con 4 opciones (A-D), solo 1 correcta.
- No incluye las respuestas: úsalo para practicar de cara al cuestionario. TODAS las preguntas del cuestionario saldrán de este banco de preguntas.

Bloque 1

Fundamentos y cultura DevOps

Pregunta 1

Un equipo de desarrollo prioriza sacar features rápido y un equipo de operaciones prioriza mantener los sistemas estables. ¿Qué nombre recibe el enfoque que busca romper esa barrera y hacer que ambos equipos colaboren?

- A) Cultura DevOps
- B) Programación orientada a objetos
- C) Arquitectura cliente-servidor
- D) Virtualización de hardware

Pregunta 2

¿Cuál de las siguientes afirmaciones sobre Docker y DevOps es correcta?

- A) Docker es sinónimo de DevOps, son exactamente lo mismo
- B) Docker ayuda a lograr una cultura DevOps, pero no es DevOps en sí mismo
- C) DevOps es una versión antigua de Docker
- D) Docker sustituye la necesidad de que Dev y Ops colaboren

Pregunta 3

¿Cuál es el motivo típico por el que una aplicación "funciona en mi máquina" pero falla en el servidor?

- A) El servidor tiene menos memoria RAM siempre
- B) El código fuente se corrompe al subirlo
- C) Diferencias de entorno: versión de lenguaje, librerías o configuración distintas entre máquinas
- D) El servidor no tiene conexión a internet

Pregunta 4

¿Qué lectura se recomienda para profundizar en la ciencia detrás de DevOps y el desarrollo Lean?

- A) "Clean Code", de Robert C. Martin
- B) "The Docker Book", de James Turnbull
- C) "The Phoenix Project", de Gene Kim
- D) "Accelerate", de la Dra. Nicole Forsgren

Pregunta 5

¿Qué aporta fundamentalmente Docker en el ciclo de desarrollo-despliegue?

- A) Que el mismo entorno se reproduzca igual en desarrollo, pruebas y producción
- B) Que el código se ejecute más rápido automáticamente
- C) Que ya no haga falta control de versiones
- D) Que la aplicación consuma menos ancho de banda de red

Pregunta 6

¿Qué es, estrictamente, Docker?

- A) Un sistema operativo alternativo a Linux
- B) Una plataforma para empaquetar, distribuir y ejecutar aplicaciones en contenedores
- C) Un lenguaje de programación compilado
- D) Un gestor de bases de datos

Pregunta 7

Según lo visto en clase, ¿qué objetivos suelen entrar en conflicto entre el equipo de desarrollo y el de operaciones?

- A) El idioma que usan para comunicarse
- B) El horario de trabajo de cada equipo
- C) Entregar funcionalidades rápido (Dev) frente a mantener sistemas estables (Ops)
- D) El sistema operativo de sus portátiles personales

Pregunta 8

¿En qué año nació Docker como proyecto de código abierto?

- A) 2005
- B) 2001
- C) 2018
- D) 2013

Pregunta 9

¿Cuál de estas NO es una ventaja real de usar Docker para desplegar una aplicación?

- A) Mejora automáticamente el algoritmo de la aplicación sin tocar el código
- B) Portabilidad entre máquinas con Docker instalado
- C) Despliegues más rápidos y reproducibles
- D) Aislamiento entre aplicaciones en la misma máquina

Pregunta 10

¿Qué papel juegan las metodologías ágiles y la cultura DevOps frente al conflicto Dev vs Ops?

- A) Eliminan por completo la necesidad del equipo de operaciones
- B) Son la solución habitual para reducir esa fricción y mejorar la colaboración entre ambos equipos
- C) Solo aplican a proyectos que no usan contenedores
- D) Sustituyen completamente el uso de Docker

Bloque 2

Contenedores vs. máquinas virtuales

Pregunta 11

¿Qué virtualiza exactamente una máquina virtual tradicional (VirtualBox, VMware...)?

- A) Solo la tarjeta de red
- B) Solo el sistema de ficheros
- C) El hardware completo, mediante un hipervisor que reparte los recursos físicos entre sistemas operativos invitados completos
- D) Nada, una VM y un contenedor son técnicamente idénticos

Pregunta 12

¿Por qué un contenedor Docker arranca mucho más rápido que una máquina virtual?

- A) Porque usa más memoria RAM que una VM
- B) Porque se ejecuta siempre en la nube
- C) Porque no tiene sistema de ficheros propio
- D) Porque comparte el kernel del sistema operativo host en lugar de arrancar un SO invitado completo

Pregunta 13

En la comparativa VM vs contenedor vista en clase, ¿qué afirmación es correcta sobre el aislamiento?

- A) Las VMs ofrecen un aislamiento más fuerte al tener un SO completo independiente; los contenedores, al compartir kernel, tienen un aislamiento algo menor pero suficiente para la mayoría de casos
- B) Los contenedores siempre están más aislados que cualquier VM
- C) Ninguno de los dos aísla realmente nada
- D) El aislamiento depende únicamente del proveedor de nube usado

Pregunta 14

¿Es técnicamente posible ejecutar varios contenedores Docker dentro de una única máquina virtual?

- A) No, Docker y las VMs son tecnologías incompatibles entre sí
- B) Sí; de hecho es una combinación muy habitual: una VM (física o en la nube) ejecutando varios contenedores dentro
- C) Solo si la máquina virtual usa Windows Server
- D) Solo si se desactiva la virtualización de hardware

Pregunta 15

¿Qué escenario encajaría mejor con una máquina virtual que con un contenedor Docker?

- A) Ejecutar una API REST sencilla con sus dependencias
- B) Levantar una base de datos para pruebas rápidas
- C) Necesitar un sistema operativo invitado completamente distinto al del host, con aislamiento total de hardware
- D) Ejecutar un script que tarda dos segundos en completarse

Pregunta 16

¿Por qué suelen caber muchos más contenedores que máquinas virtuales en un mismo servidor físico?

- A) Porque los contenedores no consumen CPU
- B) Porque las licencias de Docker son más baratas
- C) Porque los contenedores no necesitan red
- D) Porque cada contenedor no necesita un sistema operativo completo propio, a diferencia de cada VM

Pregunta 17

¿Qué gestiona un hipervisor en el contexto de las máquinas virtuales?

- A) El reparto del hardware físico entre los distintos sistemas operativos invitados
- B) Únicamente la conexión a internet de las VMs
- C) El registro de imágenes Docker
- D) La compilación del código fuente de la aplicación

Pregunta 18

Un contenedor y una VM ejecutan la misma aplicación web. En igualdad de condiciones, ¿cuál consumirá previsiblemente menos disco y RAM?

- A) La máquina virtual, porque está más optimizada por diseño
- B) El contenedor, al no incluir un sistema operativo invitado completo
- C) Ambos consumen exactamente lo mismo siempre
- D) Depende exclusivamente del navegador usado para acceder a la app

Pregunta 19

¿Qué parte del sistema NO necesita arrancar un contenedor Docker, a diferencia de una VM?

- A) El proceso de la aplicación
- B) La red del host
- C) Un sistema operativo invitado completo con su propio kernel
- D) El propio Docker Engine

Bloque 3

Arquitectura de Docker y registries

Pregunta 20

¿Qué es, exactamente, el "Docker daemon" (`dockerd`)?

- A) El comando que el usuario escribe en la terminal
- B) Una imagen oficial de Docker Hub
- C) El nombre comercial de Docker Desktop
- D) El proceso en segundo plano que gestiona realmente contenedores, imágenes, redes y volúmenes

Pregunta 21

¿Cómo se comunican normalmente el Docker client y el Docker daemon?

- A) A través de una API, típicamente mediante un socket local o sobre red
- B) Intercambiando ficheros de texto plano manualmente
- C) No se comunican entre sí, funcionan de forma independiente
- D) Únicamente a través de Docker Hub

Pregunta 22

¿Qué es un "registry" en el ecosistema Docker?

- A) Un fichero de configuración del sistema operativo host
- B) Un servidor donde se almacenan y distribuyen imágenes Docker, como Docker Hub
- C) Un tipo especial de contenedor que nunca se detiene
- D) El historial de comandos ejecutados por el usuario

Pregunta 23

Ejecutas `docker run miapp:2.0` y esa imagen no está descargada localmente. ¿Qué ocurre?

- A) Docker da un error inmediato y no continúa
- B) Se crea un contenedor vacío sin ninguna imagen real
- C) El daemon la descarga automáticamente desde el registry configurado (por defecto, Docker Hub) y después crea el contenedor
- D) Hay que descargarla manualmente con un navegador antes de poder usarla

Pregunta 24

¿Puede el Docker client conectarse a un daemon que se ejecuta en una máquina distinta?

- A) No, cliente y daemon deben estar siempre en la misma máquina
- B) No, Docker no soporta ese tipo de arquitectura
- C) Solo si ambas máquinas usan macOS
- D) Sí, es posible configurar el cliente para hablar con un daemon remoto a través de la red

Pregunta 25

¿Qué alternativas a Docker Hub existen como registries de imágenes?

- A) Otros registries, públicos o privados, como GitHub Container Registry o Amazon ECR
- B) Ninguna, Docker Hub es el único registry que existe en el mundo
- C) GitHub y GitLab son en realidad registries de imágenes, no de código
- D) Los registries solo sirven para almacenar código fuente, nunca imágenes

Pregunta 26

¿Por qué una empresa podría preferir montar un registry privado en lugar de usar Docker Hub público?

- A) Porque Docker Hub no permite nunca descargar imágenes
- B) Para evitar que imágenes con información interna sean accesibles fuera de la organización
- C) Porque los registries privados no requieren autenticación
- D) No existe ninguna razón real para hacerlo

Pregunta 27

¿Qué necesita, en el fondo, el Docker daemon en Linux para poder ejecutar contenedores?

- A) Una licencia comercial activa
- B) Una conexión permanente a Docker Hub
- C) Un kernel Linux (en Windows/macOS se usa una capa de virtualización ligera para proporcionarlo)
- D) Un hipervisor de tipo 1 obligatoriamente

Pregunta 28

¿Cuál de estas afirmaciones describe mejor el conjunto "Docker Engine"?

- A) Es solo el nombre comercial de Docker Hub
- B) Es un tipo de volumen persistente
- C) Es un lenguaje de scripting exclusivo de Docker
- D) Es el motor completo formado por el daemon, una API y el cliente de línea de comandos

Bloque 4

Docker Hub e imágenes de la asignatura

Pregunta 29

¿Qué es Docker Hub?

- A) El registro público oficial de Docker, donde se publican y descargan imágenes
- B) Un IDE pensado para programar dentro de contenedores
- C) Un comando de la CLI de Docker
- D) Un tipo de volumen compartido entre contenedores

Pregunta 30

¿Hace falta tener una cuenta en Docker Hub para descargar una imagen pública como `nginx` ?

- A) Sí, siempre es obligatorio iniciar sesión primero
- B) No, las imágenes públicas pueden descargarse sin cuenta ni `docker login`
- C) Solo si la imagen pesa más de 500 MB
- D) Solo los días laborables

Pregunta 31

¿Qué significa que una imagen en Docker Hub se llame simplemente `redis` en vez de `usuario/redis` ?

- A) Que pertenece a un usuario personal llamado 'redis'
- B) Que está obsoleta y no recibe actualizaciones
- C) Que es una imagen oficial, publicada en el namespace raíz de Docker Hub
- D) Que es de pago

Pregunta 32

De las imágenes que se usarán en esta asignatura, ¿cuál es una base de datos documental (NoSQL)?

- A) `nginx`
- B) `redis`
- C) `mariadb`
- D) `mongo`

Pregunta 33

¿Para qué hace falta sí o sí tener cuenta en Docker Hub y ejecutar `docker login` ?

- A) Para subir (`docker push`) tus propias imágenes a tu repositorio
- B) Para descargar cualquier imagen pública existente
- C) Para ejecutar `docker run` por primera vez
- D) Para instalar Docker en tu máquina

Pregunta 34

De las imágenes curadas para la asignatura, ¿cuál usarías como runtime para ejecutar código PHP?

- A) mariadb
- B) php
- C) redis
- D) nginx

Pregunta 35

¿Qué suele indicar, al buscar una imagen en Docker Hub, un número alto de "pulls" junto con la insignia de "oficial"?

- A) Que la imagen contiene malware con alta probabilidad
- B) Que pesa más de lo normal
- C) Que es una imagen ampliamente usada y considerada de confianza
- D) Que solo es gratuita el primer mes

Pregunta 36

¿Cuál de estas imágenes de la asignatura usarías típicamente como servidor web para servir contenido estático?

- A) `mongo`
- B) `redis`
- C) `mariadb`
- D) `nginx`

Bloque 5

Imagen vs. contenedor

Pregunta 37

¿Cuál es la diferencia fundamental entre una imagen Docker y un contenedor?

- A) La imagen es una plantilla de solo lectura; el contenedor es una instancia en ejecución (o parada) creada a partir de ella
- B) Son términos sinónimos, se usan indistintamente sin ninguna diferencia técnica
- C) Una imagen solo puede generar un único contenedor en toda su vida útil
- D) El contenedor se almacena en Docker Hub y la imagen nunca

Pregunta 38

Si modificas un fichero dentro de un contenedor en ejecución, ¿qué le ocurre a la imagen original de la que partió?

- A) La imagen cambia automáticamente para reflejar ese cambio
- B) No se modifica: el cambio queda en la capa escribible propia del contenedor
- C) La imagen se bloquea y ya no se puede reutilizar
- D) Se crea una nueva imagen automáticamente con cada cambio

Pregunta 39

Ejecutas `docker run nginx:1.25` tres veces seguidas sin usar `--rm` ni eliminar nada. ¿Qué ocurre?

- A) La segunda y tercera vez dan error porque la imagen ya está en uso
- B) Se actualiza el mismo contenedor cada vez, nunca se crean nuevos
- C) Se crean tres contenedores distintos e independientes, todos a partir de la misma imagen `nginx:1.25`
- D) Solo se puede ejecutar un contenedor por imagen a la vez

Pregunta 40

¿Qué pasa con un contenedor parado (no eliminado)? ¿Sigue ocupando espacio en disco?

- A) No, un contenedor parado desaparece automáticamente del disco
- B) Se convierte automáticamente en una nueva imagen
- C) Se mueve a una papelera temporal en Docker Hub
- D) Sí, sigue existiendo y ocupando espacio hasta que se elimina explícitamente con

```
docker rm
```

Pregunta 41

¿Cuántos contenedores distintos se pueden crear, como máximo, a partir de una misma imagen?

- A) Tantos como se quiera: cada uno es una instancia independiente de la misma plantilla
- B) Solo uno
- C) Como máximo dos, por diseño de Docker
- D) Ninguno, las imágenes no son ejecutables directamente

Pregunta 42

¿Qué identifica de forma única a un contenedor concreto frente a otros creados de la misma imagen?

- A) El nombre de la imagen de la que proviene
- B) Su propio ID de contenedor (y, opcionalmente, un nombre asignado)
- C) El tag de la imagen base
- D) La versión del Docker Engine instalada

Pregunta 43

¿Qué comando elimina de forma definitiva un contenedor ya parado?

- A) `docker rmi`
- B) `docker stop`
- C) `docker rm`
- D) `docker delete`

Pregunta 44

Tienes `app:1.0` y `app:2.0` construidas en momentos distintos a partir de Dockerfiles diferentes. ¿Qué es cierto sobre ellas?

- A) Son la misma imagen con dos nombres distintos sin ninguna diferencia real
- B) `app:2.0` sustituye automáticamente a `app:1.0`, que desaparece del sistema
- C) No pueden coexistir nunca en la misma máquina
- D) Son imágenes potencialmente distintas, identificadas por el mismo repositorio pero distinto tag, y pueden convivir sin problema

Bloque 6

PID1 y ciclo de vida de un contenedor

Pregunta 45

En Unix/Linux, ¿qué es el PID1?

- A) El primer proceso que arranca el sistema, del que derivan todos los demás
- B) El identificador de la tarjeta de red principal
- C) El último proceso en terminar al apagar el sistema
- D) Un proceso exclusivo de los contenedores, no existe en Linux normal

Pregunta 46

En el contexto de un contenedor Docker, ¿qué significa la fórmula "contenedor = imagen + PID1"?

- A) Que la imagen y el PID1 son el mismo concepto
- B) Que un contenedor es una imagen ejecutándose, donde el comando lanzado se convierte en el proceso principal (PID1) de ese contenedor
- C) Que PID1 es el nombre que recibe siempre la imagen base
- D) Que cada imagen solo puede tener un PID1 fijo predefinido por Docker Hub

Pregunta 47

Ejecutas `docker run -it ubuntu:22.04 bash` y luego escribes `exit` dentro. ¿Qué ocurre con el contenedor?

- A) Sigue ejecutándose en segundo plano indefinidamente
- B) Docker lo reinicia automáticamente
- C) El contenedor se detiene, porque `bash` (su PID1) ha terminado
- D) No pasa nada, `exit` solo cierra la terminal del host

Pregunta 48

¿Por qué `docker run ubuntu:22.04` (sin `-it` ni un comando que se quede esperando) suele terminar casi al instante?

- A) Porque Docker detecta un error de permisos
- B) Porque Ubuntu necesita licencia para ejecutarse en contenedores
- C) Porque la imagen está corrupta
- D) Porque, sin un proceso en primer plano que lo mantenga activo, el PID1 por defecto termina enseguida y el contenedor se para con él

Pregunta 49

¿Qué ocurre si el proceso PID1 de un contenedor termina de forma inesperada (p. ej. por un error no controlado)?

- A) El contenedor se detiene, igual que si hubiera terminado de forma normal
- B) El contenedor sigue funcionando porque Docker crea un PID1 de reemplazo automáticamente
- C) Solo se detiene si se usó la opción `--rm`
- D) El contenedor pasa a estado "Paused" en vez de "Exited"

Pregunta 50

¿Cuáles son, a grandes rasgos, los estados por los que pasa un contenedor en su ciclo de vida?

- A) Solo 'encendido' y 'apagado', como un interruptor
- B) Creado, en ejecución (running), parado (exited) y, finalmente, eliminado
- C) Únicamente 'running', no existen más estados
- D) Los contenedores no tienen estados, se consideran todos iguales

Pregunta 51

¿Qué diferencia hay entre `docker stop` y `docker kill`?

- A) Son exactamente lo mismo, sin ninguna diferencia práctica
- B) `docker kill` solo puede aplicarse sobre imágenes, nunca sobre contenedores
- C) `docker stop` intenta una parada ordenada dando un margen de tiempo; `docker kill` detiene el contenedor de forma inmediata y brusca
- D) `docker stop` elimina el contenedor además de pararlo, y `docker kill` no

Pregunta 52

¿Qué diferencia hay entre `docker run` y `docker start` ?

- A) Son términos sinónimos, intercambiables en cualquier contexto
- B) `docker start` solo funciona con imágenes, nunca con contenedores ya creados
- C) `docker run` nunca descarga imágenes desde un registry
- D) `docker run` crea un contenedor nuevo (descargando la imagen si hace falta);
`docker start` vuelve a arrancar un contenedor ya existente que estaba parado, sin crear uno nuevo

Pregunta 53

¿Qué sentido pedagógico tiene usar `docker run --rm -it imagen sh` para hacer pruebas rápidas?

- A) Evita acumular contenedores parados innecesarios una vez termina la prueba, ya que se autoelimina al salir
- B) Hace que el contenedor nunca pueda detenerse
- C) Impide revisar los logs del contenedor mientras está vivo
- D) Obliga a usar siempre la imagen `latest`

Pregunta 54

Un contenedor aparece en `docker ps -a` como 'Exited (137)'. ¿Qué representa ese número entre paréntesis?

- A) El tamaño de la imagen en megabytes
- B) El código de salida del proceso principal (PID1) del contenedor
- C) El número de puertos publicados
- D) La versión de Docker instalada en el host

Bloque 7

docker run y sus opciones

Pregunta 55

¿Qué hace la opción `-d` en `docker run -d nginx:1.25` ?

- A) Descarga la imagen sin crear contenedor
- B) Elimina el contenedor automáticamente al terminar
- C) Ejecuta el contenedor en segundo plano (modo 'detached'), devolviendo el control a la terminal
- D) Activa el modo de depuración detallado

Pregunta 56

Si no indicas `--name` al ejecutar `docker run`, ¿qué hace Docker?

- A) Rechaza la ejecución hasta que se indique un nombre
- B) Usa siempre el mismo nombre genérico 'container1' para todos
- C) Deja el contenedor sin ningún identificador legible
- D) Le asigna automáticamente un nombre aleatorio (tipo adjetivo + apellido de científico, p. ej. 'silly_einstein')

Pregunta 57

¿Qué combinación de opciones se usa típicamente para abrir una sesión de terminal interactiva dentro de un contenedor recién creado?

- A) `-it` , junto con `bash` o `sh` como comando
- B) `-d` , sola
- C) `--rm` , sola
- D) `-p 80:80`

Pregunta 58

¿Qué hacen exactamente las opciones `-i` y `-t` por separado?

- A) `-i` publica un puerto y `-t` asigna un nombre
- B) `-i` mantiene abierta la entrada estándar (STDIN) y `-t` asigna una pseudo-terminal; juntas (`-it`) dan una sesión usable
- C) Ambas sirven exclusivamente para depurar la red del contenedor
- D) `-i` elimina el contenedor al salir y `-t` lo pausa

Pregunta 59

¿Qué ocurre si ejecutas dos veces `docker run --name web nginx:1.25` seguidas, sin eliminar el primer contenedor?

- A) Se crean dos contenedores distintos con el mismo nombre sin ningún problema
- B) El segundo sobrescribe silenciosamente al primero
- C) La segunda ejecución da error, porque ya existe un contenedor llamado `web`
- D) Docker renombra automáticamente el segundo contenedor

Pregunta 60

¿Qué hace `docker run --rm alpine:3.19` ?

- A) Elimina la imagen `alpine` del sistema tras usarla
- B) Ejecuta el contenedor en modo de solo lectura total
- C) Bloquea cualquier conexión de red del contenedor
- D) Elimina automáticamente el contenedor en cuanto termina su ejecución

Pregunta 61

Quieres publicar el puerto 80 de un contenedor nginx en el puerto 8080 de tu máquina.
¿Qué opción de `docker run` usarías?

- A) `-p 8080:80`
- B) `-p 80:8080`
- C) `--port=8080`
- D) `-d 8080:80`

Pregunta 62

¿Qué pasa si intentas publicar en el host un puerto que ya está siendo usado por otro contenedor (p. ej. `-p 8080:80` dos veces)?

- A) Docker reasigna automáticamente otro puerto libre sin avisar
- B) El segundo `docker run` falla con un error de puerto ya en uso
- C) Ambos contenedores comparten el puerto sin ningún conflicto
- D) El primer contenedor se detiene automáticamente para dejar sitio al segundo

Pregunta 63

¿Cuál es la diferencia entre `docker run -it imagen bash` y `docker exec -it contenedor bash` ?

- A) Son exactamente equivalentes en todos los casos
- B) `docker exec` solo funciona sobre imágenes, nunca sobre contenedores
- C) `docker run` crea un contenedor nuevo; `docker exec` abre una sesión dentro de un contenedor que ya existe y está en ejecución
- D) `docker run -it` nunca puede usarse con `bash`

Pregunta 64

¿Qué parámetro de `docker run` define variables de entorno disponibles dentro del contenedor?

- A) `-v VAR=valor`
- B) `-p VAR=valor`
- C) `--name VAR=valor`
- D) `-e VAR=valor`

Pregunta 65

Lanzas `docker run -d --name web -p 8080:80 nginx:1.25` y después cierras la terminal donde lo lanzaste. ¿Sigue funcionando el contenedor?

- A) Sí, porque se lanzó en modo 'detached' (`-d`), independiente de la terminal que lo inició
- B) No, se detiene en cuanto se cierra esa terminal
- C) Solo sigue funcionando si además se usó `--rm`
- D) Depende exclusivamente del sistema operativo del host

Pregunta 66

¿Para qué sirve específicamente la opción `--name` de `docker run` ?

- A) Para cambiar el nombre de la imagen de origen de forma permanente
- B) Para asignar al contenedor un nombre fácil de recordar y referenciar, en vez del nombre aleatorio por defecto
- C) Para renombrar Docker Hub
- D) Para indicar el nombre del Dockerfile a usar

Bloque 8

docker ps y docker logs

Pregunta 67

¿Qué diferencia hay entre `docker ps` y `docker ps -a` ?

- A) Son exactamente lo mismo
- B) `docker ps -a` solo muestra contenedores parados, nunca los que están en marcha
- C) `docker ps` solo muestra los contenedores en ejecución; `docker ps -a` muestra también los parados
- D) `docker ps -a` muestra únicamente las imágenes, no los contenedores

Pregunta 68

Un compañero ejecuta `docker run nginx:1.25` (sin `-d`) y, al abrir otra terminal y mirar `docker ps`, no ve ningún contenedor. ¿Cuál es la explicación más probable?

- A) Docker está roto y hay que reinstalarlo
- B) `nginx` no es una imagen válida
- C) Hace falta reiniciar el ordenador para que aparezca
- D) Al no usar `-d`, el contenedor ocupa la terminal en primer plano; si lo paró con Ctrl+C habría terminado, y si sigue vivo debería aparecer en `docker ps` de la otra terminal salvo que algo lo haya detenido

Pregunta 69

¿Qué comando muestra la salida estándar (stdout/stderr) generada por un contenedor?

- A) `docker logs nombre_contenedor`
- B) `docker ps nombre_contenedor`
- C) `docker show nombre_contenedor`
- D) `docker output nombre_contenedor`

Pregunta 70

¿Qué hace la opción `-f` (follow) en `docker logs -f miapp` ?

- A) Fuerza la eliminación inmediata del contenedor
- B) Muestra los logs en tiempo real a medida que se generan, de forma similar a `tail -f`
- C) Filtra y muestra solo las líneas de error
- D) Formatea automáticamente los logs como JSON

Pregunta 71

Un contenedor web se ha parado de forma inesperada. ¿Por qué es buena idea revisar `docker logs` antes que nada?

- A) Los logs desaparecen en cuanto el contenedor se detiene, así que no sirve de nada
- B) Los logs solo contienen información de red, irrelevante para este caso
- C) Normalmente contienen el motivo del fallo (excepciones, errores de arranque...) que causó la parada
- D) Revisar los logs reinicia automáticamente el contenedor

Pregunta 72

¿Qué opción de `docker logs` te permite ver solo las últimas 50 líneas en vez de todo el histórico?

- A) `--last 50`
- B) `-n50`
- C) `--lines=50`
- D) `--tail 50`

Pregunta 73

¿En qué columna de la salida de `docker ps` verías algo como 'Up 10 minutes' o 'Exited (1) 2 minutes ago'?

- A) STATUS
- B) IMAGE
- C) PORTS
- D) COMMAND

Pregunta 74

Tu contenedor `web` aparece como 'Up' en `docker ps`, pero el navegador no carga nada en `http://localhost:8080`. ¿Qué sería razonable revisar primero?

- A) Reinstalar Docker directamente
- B) Los logs del contenedor y si el puerto se publicó correctamente con `-p`
- C) Cambiar el sistema operativo del host
- D) Es imposible que esto ocurra si el estado es 'Up'

Pregunta 75

¿Qué comando mostraría solo los IDs de los contenedores en ejecución, útil para combinarlo con otros comandos (p. ej. para pararlos todos de golpe)?

- A) `docker ps --only-id`
- B) `docker ps -i`
- C) `docker ps -q`
- D) `docker ps --ids`

Pregunta 76

¿Qué comando te diría rápidamente qué puertos tiene publicados un contenedor concreto en este momento?

- A) `docker logs contenedor`
- B) `docker images`
- C) `docker pull contenedor`
- D) `docker ps` (columna PORTS) o `docker port contenedor`

Bloque 9

docker images, docker pull y tags

Pregunta 77

¿Qué diferencia hay entre `docker pull imagen` y `docker run imagen` ?

- A) `docker pull` solo descarga la imagen; `docker run` la descarga (si falta) y además crea y arranca un contenedor
- B) Son exactamente equivalentes
- C) `docker run` nunca descarga nada, solo usa lo que ya hay en disco
- D) `docker pull` crea un contenedor parado automáticamente

Pregunta 78

Si no indicas ningún tag al construir o ejecutar una imagen (p. ej. `ubuntu` en vez de `ubuntu:22.04`), ¿qué tag se usa por defecto?

- A) `stable`
- B) `latest`
- C) `last`
- D) `default`

Pregunta 79

¿Por qué se recomienda evitar el tag `latest` y fijar siempre una versión concreta como `ubuntu:22.04` ?

- A) Porque `latest` nunca funciona correctamente
- B) Porque `latest` solo existe para imágenes privadas
- C) Porque `latest` es solo una etiqueta más que puede apuntar a una versión distinta con el tiempo, rompiendo la reproducibilidad
- D) Porque usar `latest` obliga a pagar una licencia

Pregunta 80

Haces `docker pull nginx:1.25` dos veces seguidas. La segunda vez es casi instantánea. ¿Por qué?

- A) Es pura coincidencia, no hay motivo técnico
- B) Docker Hub da prioridad a la segunda petición del mismo usuario
- C) La segunda vez se descarga una versión recortada
- D) La imagen ya está descargada localmente y Docker detecta que no hace falta volver a bajarla

Pregunta 81

¿Qué comando elimina una imagen local por su nombre o ID?

- A) `docker rmi`
- B) `docker rm`
- C) `docker delete-image`
- D) `docker clean`

Pregunta 82

Intentas borrar con `docker rmi` una imagen que todavía está siendo usada por un contenedor existente (aunque esté parado). ¿Qué ocurre?

- A) Se borra sin ningún problema siempre
- B) Docker da un error; antes hay que eliminar los contenedores que dependen de ella (o forzar con la opción adecuada)
- C) Se elimina automáticamente el contenedor asociado sin avisar
- D) Se crea silenciosamente una copia de la imagen

Pregunta 83

En una línea de `docker images` como `nginx 1.25 abc123def456 2 days ago 187MB`, ¿qué representa `1.25` ?

- A) El tamaño de la imagen
- B) El identificador único de la imagen
- C) El tag, es decir, la versión o variante concreta de esa imagen
- D) El número de contenedores creados a partir de ella

Pregunta 84

¿Qué comando muestra cuánto espacio en disco están usando en total imágenes, contenedores y volúmenes de Docker?

- A) `docker size`
- B) `docker space`
- C) `docker du`
- D) `docker system df`

Pregunta 85

Una misma imagen (mismo ID interno) tiene dos tags distintos: `mi-app:1.0` y `mi-app:latest`. Borrás solo `mi-app:latest` con `docker rmi`. ¿Qué ocurre?

- A) Solo se elimina ese tag concreto; como todavía queda el tag `mi-app:1.0` apuntando a la misma imagen, esta no desaparece del todo
- B) Se elimina la imagen completa, incluido el tag `1.0`
- C) Es imposible borrar un único tag, siempre se borran todos a la vez
- D) Docker bloquea la operación y pide reiniciar el sistema

Pregunta 86

¿Qué comando lista las imágenes descargadas o construidas localmente en tu máquina?

- A) `docker ps`
- B) `docker images`
- C) `docker ls`
- D) `docker containers`

Bloque 10

docker exec -it

Pregunta 87

¿Para qué sirve principalmente `docker exec` ?

- A) Para crear una imagen nueva a partir de un Dockerfile
- B) Para eliminar un contenedor en ejecución
- C) Para ejecutar un comando adicional dentro de un contenedor que ya está en marcha
- D) Para publicar una imagen en Docker Hub

Pregunta 88

¿Qué ocurre si sales de una sesión `docker exec -it contenedor bash` escribiendo `exit` ?

- A) El contenedor se detiene por completo, como si hubiera terminado su PID1
- B) Se elimina automáticamente el contenedor
- C) Se borra la imagen de la que proviene el contenedor
- D) Se cierra solo esa sesión de terminal; el proceso principal del contenedor sigue funcionando con normalidad

Pregunta 89

¿Se puede ejecutar un comando puntual con `docker exec` sin abrir una sesión interactiva completa (sin `-it`)?

- A) Sí, por ejemplo `docker exec miapp ls /app` ejecuta ese comando y devuelve el resultado sin abrir sesión interactiva
- B) No, `docker exec` siempre exige `-it`
- C) Solo si el contenedor está parado
- D) Solo para comandos relacionados con la red

Pregunta 90

Intentas hacer `docker exec -it` sobre un contenedor que aparece como 'Exited' en `docker ps -a`. ¿Qué ocurre?

- A) Funciona exactamente igual que si estuviera en ejecución
- B) Da error: `docker exec` requiere que el contenedor esté en ejecución
- C) Docker lo arranca automáticamente sin avisar y luego entra
- D) El contenedor pasa a convertirse en una imagen

Pregunta 91

Un contenedor usa una imagen `alpine`, que no trae `bash` instalado. ¿Qué comando alternativo suele funcionar para entrar a una sesión interactiva?

- A) `cmd`
- B) `powershell`
- C) `sh`
- D) Ninguno, es obligatorio instalar `bash` primero

Pregunta 92

¿Qué ventaja tiene usar `docker exec -it contenedor bash` para depurar un problema en vez de recrear el contenedor desde cero?

- A) Ninguna, siempre es mejor recrear el contenedor
- B) Sustituye automáticamente el Dockerfile original
- C) Construye una nueva imagen con los cambios hechos dentro
- D) Permite inspeccionar el estado interno (ficheros, procesos...) del contenedor que ya está en marcha, sin interrumpir su funcionamiento

Pregunta 93

¿Qué diferencia esencial hay entre `docker run -it imagen sh` y `docker exec -it contenedor sh` en cuanto al número de contenedores?

- A) `docker run` crea un contenedor nuevo cada vez; `docker exec` no crea ninguno, reutiliza uno existente
- B) Ambos crean siempre un contenedor nuevo
- C) Ninguno de los dos crea contenedores, solo los listan
- D) `docker exec` crea dos contenedores por cada ejecución

Pregunta 94

¿Qué comando mostraría, sin entrar dentro del contenedor, qué procesos se están ejecutando en su interior?

- A) `docker ps nombre_contenedor`
- B) `docker top nombre_contenedor`
- C) `docker exec nombre_contenedor --list`
- D) `docker inspect --procs`

Bloque 11

Preguntas de escenario

Pregunta 95

Quieres probar rápidamente una imagen `ubuntu:22.04` de forma interactiva, sin que quede ningún contenedor parado después. ¿Qué combinación de opciones usarías?

- A) `-d --name prueba`
- B) `-p 80:80`
- C) `-it --rm` (junto con `bash` o `sh` como comando)
- D) `--restart=always`

Pregunta 96

Lanzas un contenedor sin `--name` y, minutos después, necesitas pararlo, pero no recuerdas el nombre aleatorio que le asignó Docker. ¿Cómo lo localizas?

- A) Es imposible, hay que recrear el contenedor
- B) Reiniciando Docker, que lo renombra
- C) Mirando `docker images`, ahí aparecen los nombres de los contenedores
- D) Con `docker ps` (o `docker ps -a`), que muestra el nombre asignado junto con el resto de contenedores

Pregunta 97

Necesitas comprobar el valor de una variable de entorno dentro de un contenedor que ya está en ejecución, sin pararlo ni recrearlo. ¿Qué harías?

- A) `docker exec -it contenedor env` (o abrir sesión con `docker exec -it contenedor sh` y comprobarlo ahí)
- B) `docker rm` y volver a crearlo con `-e`
- C) `docker build` de nuevo con la variable añadida
- D) `docker pull` de la imagen otra vez

Pregunta 98

Quieres liberar espacio eliminando de golpe todos los contenedores parados y las imágenes que no se estén usando. ¿Qué tipo de comando usarías?

- A) `docker images --delete-all`
- B) `docker system prune` (revisando antes qué se va a borrar)
- C) `docker stop --everything`
- D) No existe ninguna forma de hacer esto de golpe

Pregunta 99

Un contenedor crítico no responde a `docker stop` tras esperar el margen de tiempo habitual. ¿Qué comando más drástico usarías como último recurso?

- A) `docker pause`
- B) `docker logs`
- C) `docker kill`
- D) `docker pull`

Pregunta 100

Resume el flujo típico visto hoy para probar una imagen de Docker Hub de forma interactiva, dejando el entorno limpio al terminar:

- A) `docker build` → `docker push` → `docker rm`
- B) `docker images` → `docker stop` → `docker start`
- C) `docker login` → `docker tag` → `docker commit`
- D) `docker pull imagen:tag` → `docker run -it --rm imagen:tag sh` → explorar y salir con `exit`

¡Suerte con el repaso!